

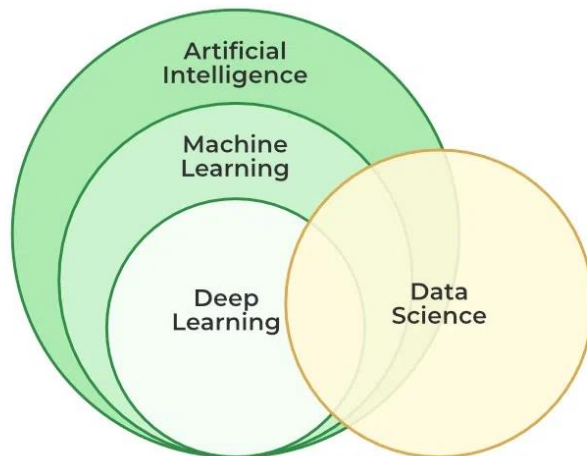
What is Deep Learning?

Deep Learning is a part of Machine Learning that uses artificial neural networks to learn from lots of data without needing explicit programming. These networks are inspired by the human brain and can be used for things like recognizing images, understanding speech, and processing language. There are different types of deep learning networks, like feedforward neural networks, convolutional neural networks, and recurrent neural networks. Deep Learning needs lots of labeled data and powerful computers to work well, but it can achieve very good results in many applications

- Artificial Neural Network
- Convolution Neural Network
- Recurrent Neural Network
- Generative Learning
- Reinforcement Learning

The definition of Deep learning is that it is the branch of machine learning that is based on artificial neural network architecture. An artificial neural network or ANN uses layers of interconnected nodes called neurons that work together to process and learn from the input data.

In a fully connected deep neural network, there is an input layer and one or more hidden layers connected one after the other. Each neuron receives input from the previous layer neurons or the input layer. The output of one neuron becomes the input to other neurons in the next layer of the network, and this process continues until the final layer produces the output of the network. The layers of the neural network transform the input data through a series of nonlinear transformations, allowing the network to learn complex representations of the input data.



Neural Network

- Biological Neurons Vs Artificial Neurons
- Single Layer Perceptron
- Multi-Layer Perceptron
- Forward and backward propagation
- Feed-forward neural networks
- Neural Network layers
- Introduction to Activation Function
- Types Of Activation Function

Difference between ANN and BNN

1. Artificial Neural Network: Artificial Neural Network (ANN) is a type of neural network that is based on a Feed-Forward strategy. It is called this because they pass information through the nodes continuously till it reaches the output node. This is also known as the simplest type of neural network.

Some advantages of ANN :

- Ability to learn irrespective of the type of data (Linear or Non-Linear).
- ANN is highly volatile and serves best in financial time series forecasting.

Some disadvantages of ANN :

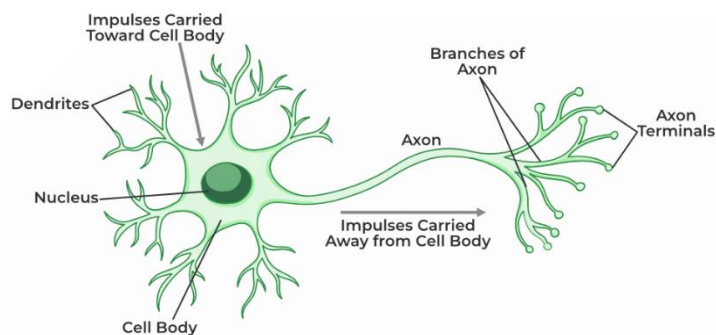
- The simplest architecture makes it difficult to explain the behavior of the network.
- This network is dependent on hardware.

2. Biological Neural Network: Biological Neural Network (BNN) is a structure that consists of Synapse, dendrites, cell body, and axon. In this neural network, the processing is carried out by neurons. Dendrites receive signals from other neurons, Soma sums all the incoming signals and axon transmits the signals to other cells.

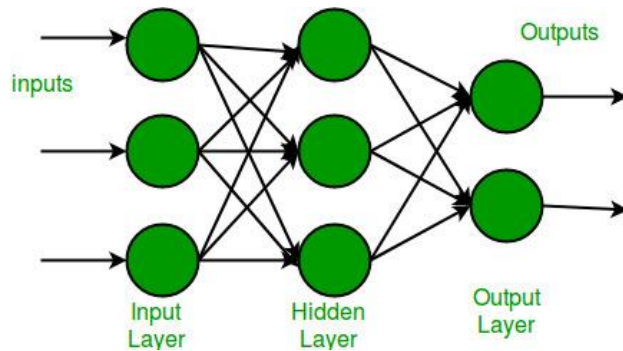
Single Layer Perceptron

Biological neuron has three basic functionality

- Receive signal from outside.
- Process the signal and enhance whether we need to send information or not.
- Communicate the signal to the target cell which can be another neuron or gland.



In the same way, neural networks also work.



What is Single Layer Perceptron?

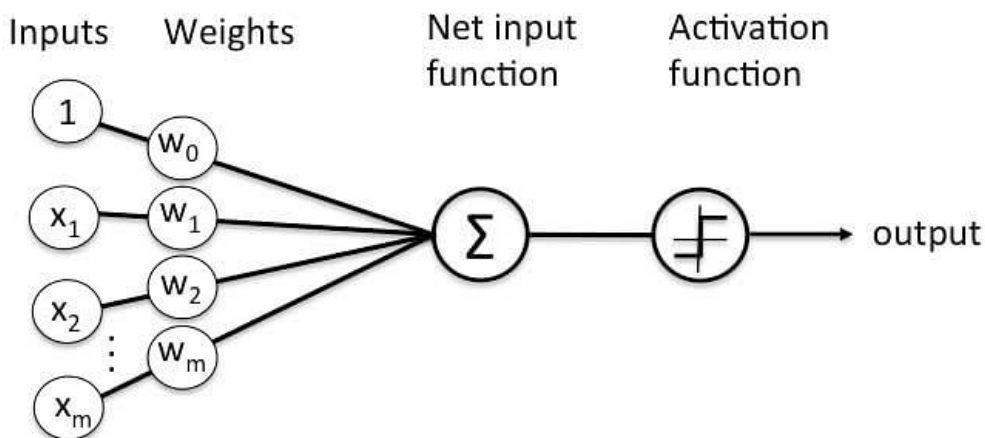
It is one of the oldest and first introduced neural networks. It was proposed by Frank Rosenblatt in 1958. Perceptron is also known as an artificial neural network. Perceptron is mainly used to compute the logical gate like AND, OR, and NOR which has binary input and binary output.

The main functionality of the perceptron is:-

Takes input from the input layer

Weight them up and sum it up.

Pass the sum to the nonlinear function to produce the output.



Components of Perceptron

Perceptron is a type of artificial neural network, which is a fundamental concept in machine learning:

1. **Input Layer:** The input layer consists of one or more input neurons, which receive input signals from the external world or from other layers of the neural network.
2. **Weights:** Each input neuron is associated with a weight, which represents the strength of the connection between the input neuron and the output neuron.
3. **Bias:** A bias term is added to the input layer to provide the perceptron with additional flexibility in modeling complex patterns in the input data.
4. **Activation Function:** The activation function determines the output of the perceptron based on the weighted sum of the inputs and the bias term. Common activation functions used in perceptrons include the step function, sigmoid function, and ReLU function.
5. **Output:** The output of the perceptron is a single binary value, either 0 or 1, which indicates the class or category to which the input data belongs.
6. **Training Algorithm:** The perceptron is typically trained using a supervised learning algorithm such as the perceptron learning algorithm or backpropagation. During training, the weights and biases of the perceptron are adjusted to minimize the error between the predicted output and the true output for a given set of training examples.
7. **Overall,** the perceptron is a simple yet powerful algorithm that can be used to perform binary classification tasks and has paved the way for more complex neural networks used in deep learning today

Types of Perceptron:

1. **Single Layer Perceptron model:** One of the easiest ANN(Artificial Neural Networks) types consists of a feed-forward network and includes a threshold transfer inside the model. The main objective of the single-layer

perceptron model is to analyze the linearly separable objects with binary outcomes. A Single-layer perceptron can learn only linearly separable patterns.

2. Multi-Layered Perceptron model: It is mainly similar to a single-layer perceptron model but has more hidden layers.

Perceptron Function

Perceptron is a function that maps its input "x," which is multiplied with the learned weight coefficient; an output value "f(x)" is generated.

$$f(x) = \begin{cases} 1 & \text{if } w \cdot x + b > 0 \\ 0 & \text{otherwise} \end{cases}$$

In the equation given above:

- "w" = vector of real-valued weights
- "b" = bias (an element that adjusts the boundary away from origin without any dependence on the input value)
- "x" = vector of input x values

$$\sum_{i=1}^m w_i x_i$$

- "m" = number of inputs to the Perceptron

The output can be represented as "1" or "0." It can also be represented as "1" or "-1" depending on which activation function is used.

Input Features: A perceptron takes multiple input features, $x_1, x_2, \dots, x_n$ each of which is assigned a weight $w_1, w_2, \dots, w_n$

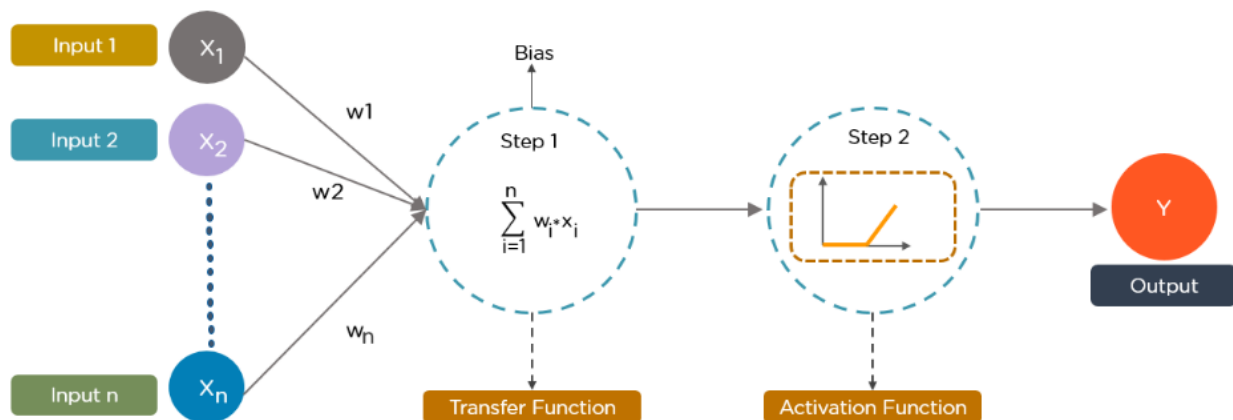
Weighted Sum: It computes the weighted sum of its inputs:

$$z = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n$$

The weighted sum z is then passed through an activation function, traditionally a step function, which decides the output of the perceptron based on whether the computed sum exceeds a threshold (typically zero).

Output: The output $\hat{y}$ of the perceptron is:

$$\hat{y} = \begin{cases} 1 & \text{if } z \geq \text{threshold} \\ 0 & \text{if } z < \text{threshold} \end{cases}$$



Activation Function:

AF is a function that is added into an ANN to help the network learn complex patterns in the data.

When comparing with a neuron-based model that is in our brains, the activation function is at the end deciding what is to be fired to the next neuron.

It takes in the output signal from the previous cell and converts it into some form that can be taken as input to the next cell.

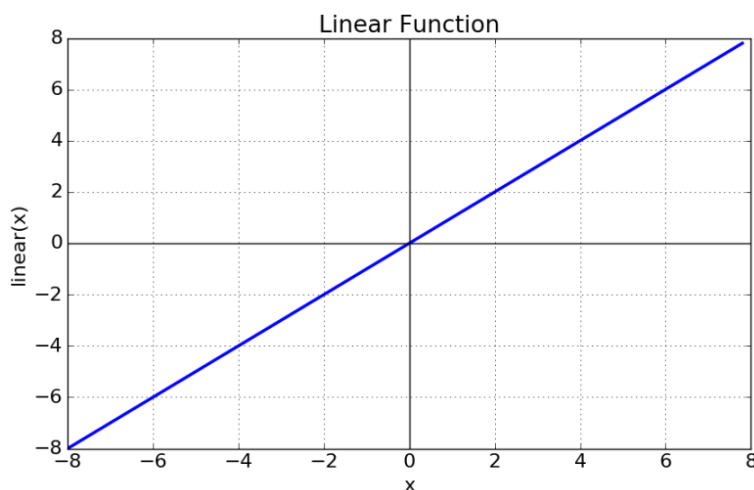
It's a non-linear transformation that we do for inputs that we receive before sending them to the next layer or neuron.

AF is also known as Transfer Function.

So basically AF can be classified into linear AF and non-linear AF.

In simple, without AF, the output signal will be a linear function (like polynomial of one degree or as a linear regression model) – they have less power to learn the complex functional mapping from data.

AF captures a non-linear relationship between the inputs and also converts the inputs to useful output.



Types of activation functions

Binary Step Function

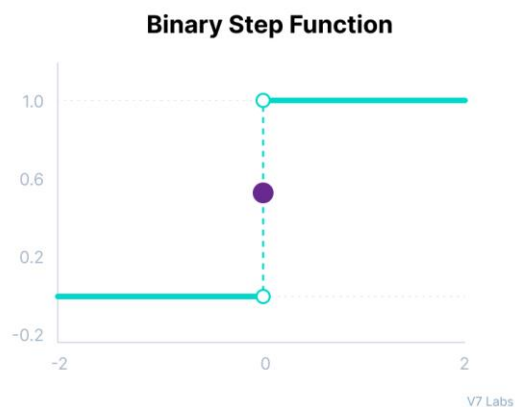
Binary step function depends on a threshold value that decides whether a neuron should be activated or not.

The first thing that comes to our mind when we have an activation function would be a threshold based classifier i.e. whether or not the neuron should be activated based on the value from the linear transformation.

In other words, if the input to the activation function is greater than a threshold, then the neuron is activated, else it is deactivated, i.e. its output is not considered for the next hidden layer. Let us look at it mathematically-

Binary step

$$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$$

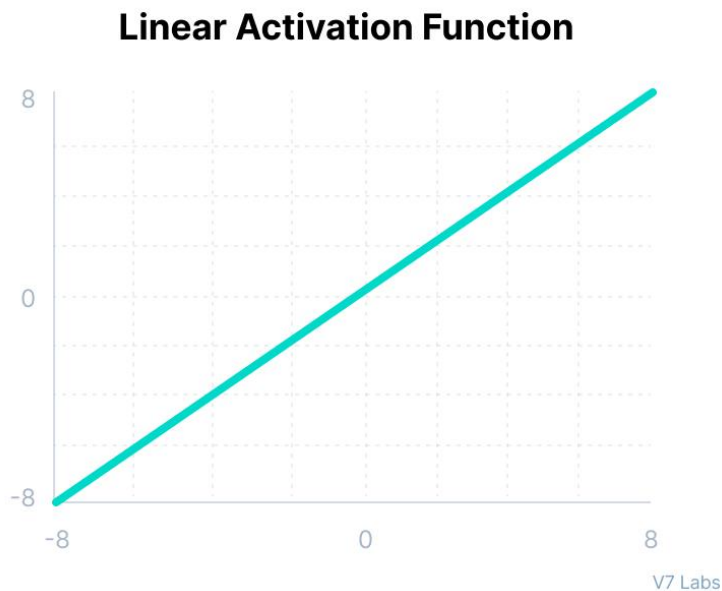


limitations of binary step function:

- It cannot provide multi-value outputs—for example, it cannot be used for multi-class classification problems.
- The gradient of the step function is zero, which causes a hindrance in the backpropagation process.

Linear Activation Function

The linear activation function, also known as "no activation," or "identity function" (multiplied x1.0), is where the activation is proportional to the input. The function doesn't do anything to the weighted sum of the input, it simply spits out the value it was given.



Mathematically it can be represented as:

Linear

$$f(x) = x$$

Non-Linear Activation Functions

The linear activation function shown linear regression model.

Because of its limited power, this does not allow the model to create complex mappings between the network's inputs and outputs.

Non-linear activation functions solve the following limitations of linear activation functions:

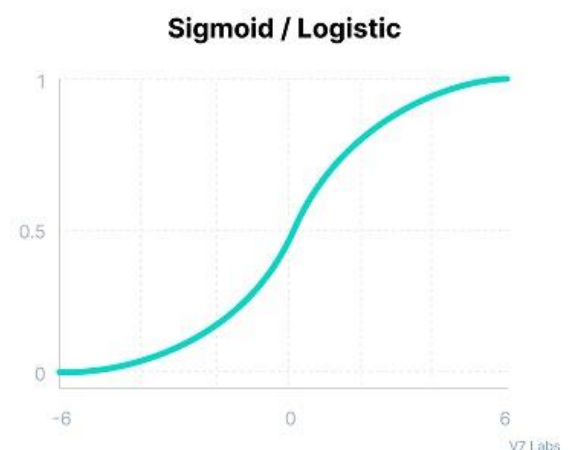
- They allow backpropagation because now the derivative function would be related to the input, and it's possible to go back and understand which weights in the input neurons can provide a better prediction.
- They allow the stacking of multiple layers of neurons as the output would now be a non-linear combination of input passed through multiple layers. Any output can be represented as a functional computation in a neural network.

Non-Linear Neural Networks Activation Functions

Sigmoid / Logistic Activation Function

function takes any real value as input and outputs values in the range of 0 to 1.

The larger the input (more positive), the closer the output value will be to 1.0, whereas the smaller the input (more negative), the closer the output will be to 0.0,



Mathematically it can be represented as:

Sigmoid / Logistic

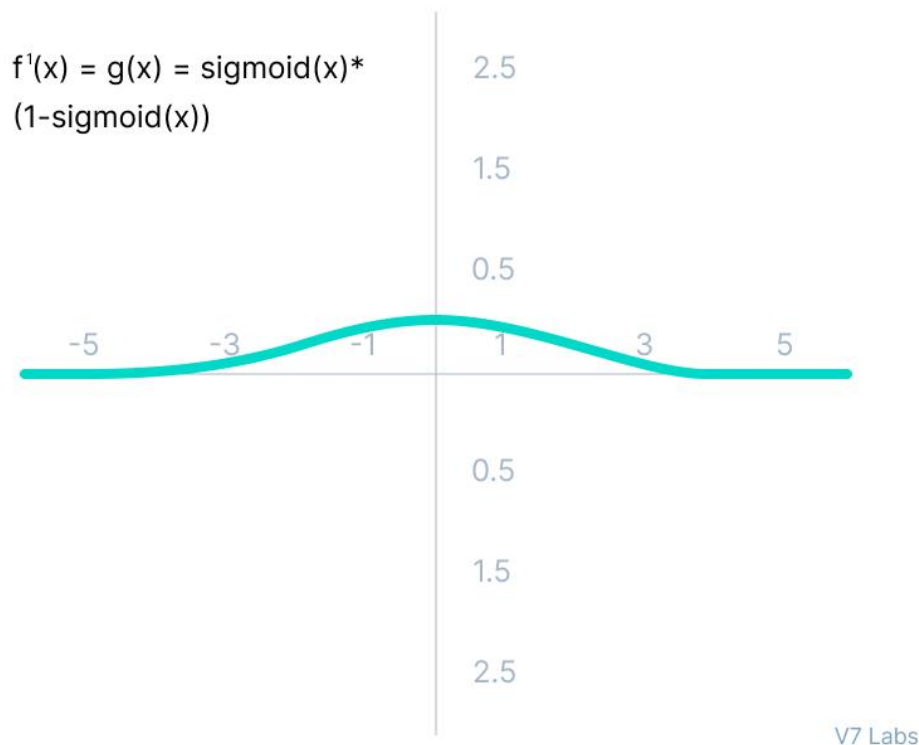
$$f(x) = \frac{1}{1 + e^{-x}}$$

sigmoid/logistic activation function is one of the most widely used functions:

- It is commonly used for models where we have to predict the probability as an output. Since probability of anything exists only between the range of 0 and 1, sigmoid is the right choice because of its range.
- The function is differentiable and provides a smooth gradient, i.e., preventing jumps in output values. This is represented by an S-shape of the sigmoid activation function.

The limitations of sigmoid function are discussed below:

- The derivative of the function is $f'(x) = \text{sigmoid}(x) * (1 - \text{sigmoid}(x))$.



As we can see from the above Figure, the gradient values are only significant for range -3 to 3, and the graph gets much flatter in other regions.

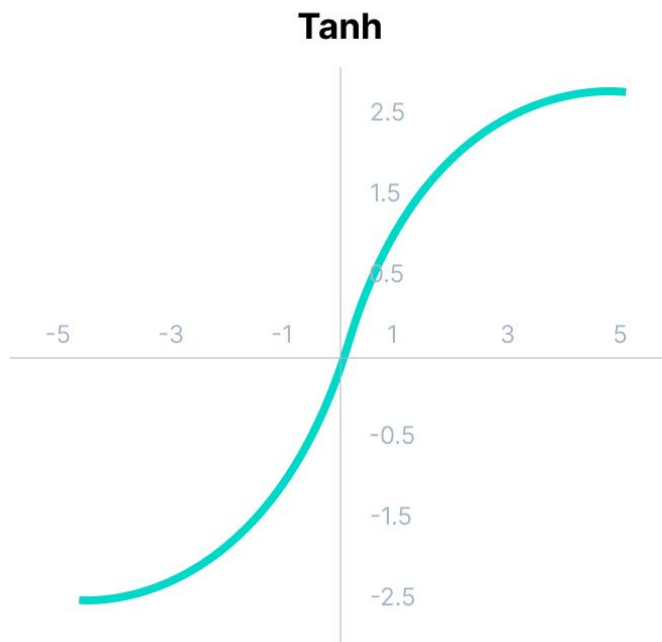
It implies that for values greater than 3 or less than -3, the function will have very small gradients. As the gradient value approaches zero, the network ceases to learn and suffers from the *Vanishing gradient* problem.

- The output of the logistic function is not symmetric around zero. So the output of all the neurons will be of the same sign. This makes the training of the neural network more difficult and unstable.

Tanh Function (Hyperbolic Tangent)

Tanh function is very similar to the sigmoid/logistic activation function, and even has the same S-shape with the difference in output range of -1 to 1.

In Tanh, the larger the input (more positive), the closer the output value will be to 1.0, whereas the smaller the input (more negative), the closer the output will be to -1.0.



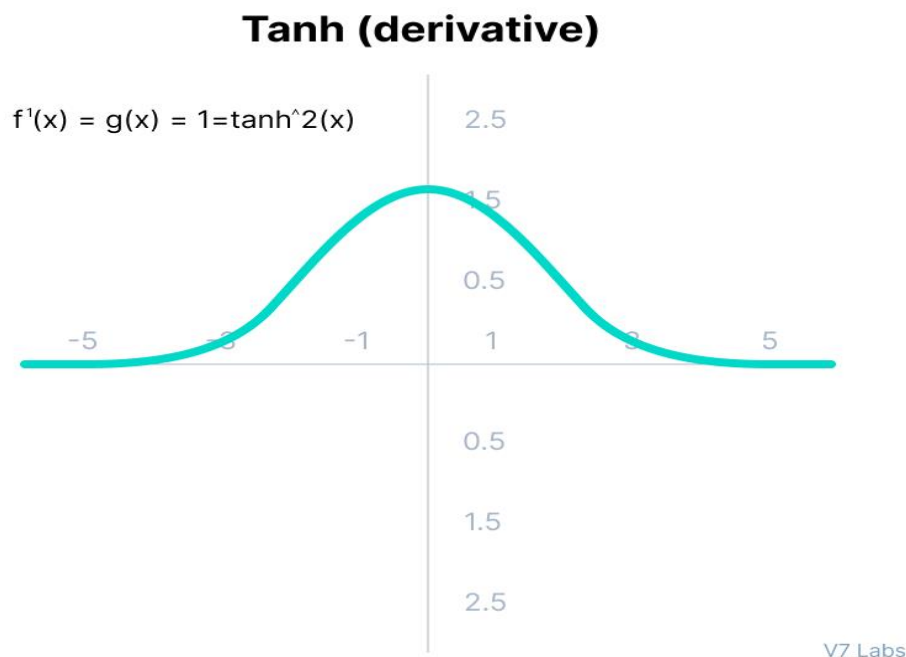
Mathematically it can be represented as:

$$\text{Tanh}$$
$$f(x) = \frac{(e^x - e^{-x})}{(e^x + e^{-x})}$$

Advantages of using this activation function are:

- The output of the tanh activation function is Zero centered; hence we can easily map the output values as strongly negative, neutral, or strongly positive.
- Usually used in hidden layers of a neural network as its values lie between -1 to 1; therefore, the mean for the hidden layer comes out to be 0 or very close to it. It helps in centering the data and makes learning for the next layer much easier.

Have a look at the gradient of the tanh activation function to understand its limitations.



As you can see— it also faces the problem of vanishing gradients similar to the sigmoid activation function. Plus the gradient of the tanh function is much steeper as compared to the sigmoid function.

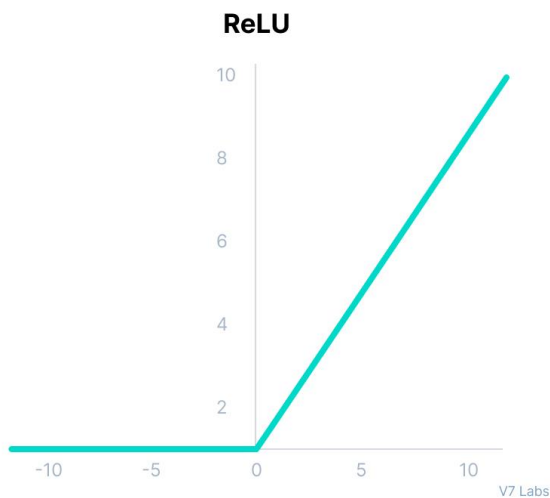
ReLU Function

ReLU stands for Rectified Linear Unit.

Although it gives an impression of a linear function, ReLU has a derivative function and allows for backpropagation while simultaneously making it computationally efficient.

The main catch here is that the ReLU function does not activate all the neurons at the same time.

The neurons will only be deactivated if the output of the linear transformation is less than 0.



Mathematically it can be represented as:

ReLU

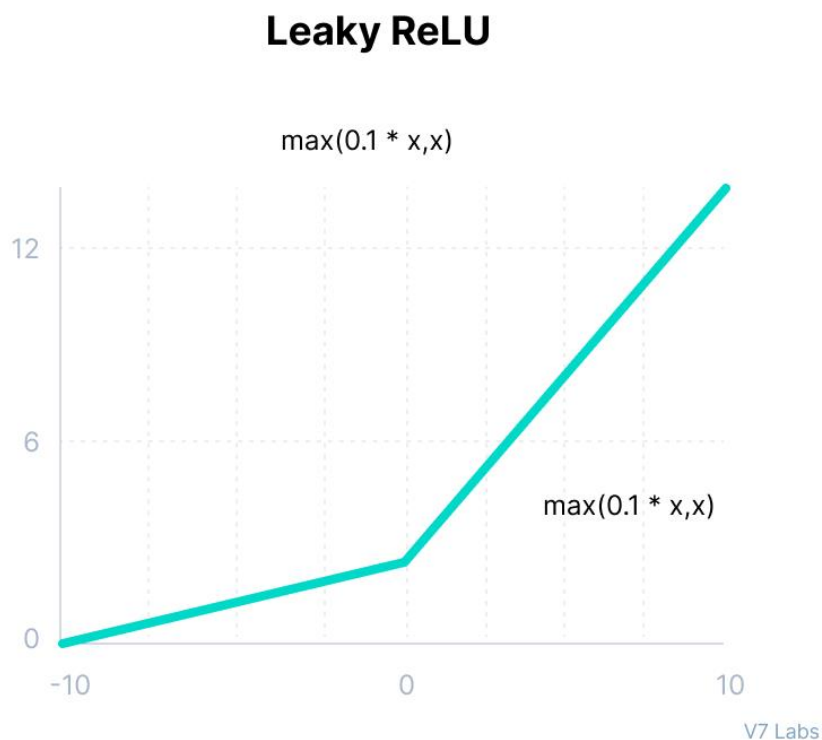
$$f(x) = \max(0, x)$$

The advantages of using ReLU as an activation function are as follows:

- Since only a certain number of neurons are activated, the ReLU function is far more computationally efficient when compared to the sigmoid and tanh functions.
- ReLU accelerates the convergence of gradient descent towards the global minimum of the loss function due to its linear, non-saturating property.

Leaky ReLU Function

Leaky ReLU is an improved version of ReLU function to solve the Dying ReLU problem as it has a small positive slope in the negative area.



Mathematically it can be represented as:

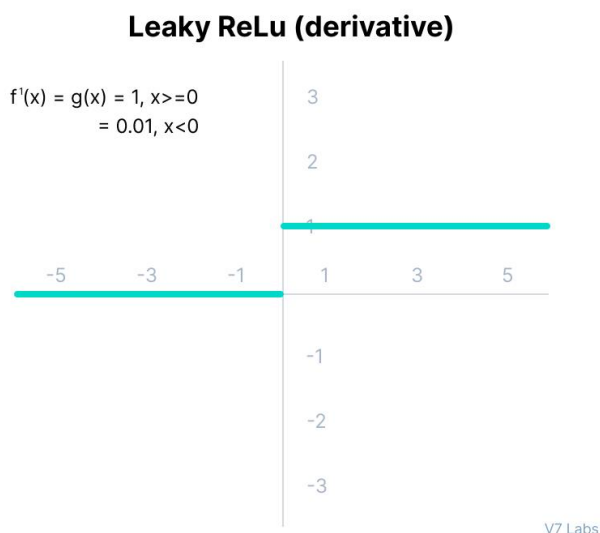
Leaky ReLU

$$f(x) = \max(0.1x, x)$$

The advantages of Leaky ReLU are same as that of ReLU, in addition to the fact that it does enable backpropagation, even for negative input values.

By making this minor modification for negative input values, the gradient of the left side of the graph comes out to be a non-zero value. Therefore, we would no longer encounter dead neurons in that region.

Here is the derivative of the Leaky ReLU function.

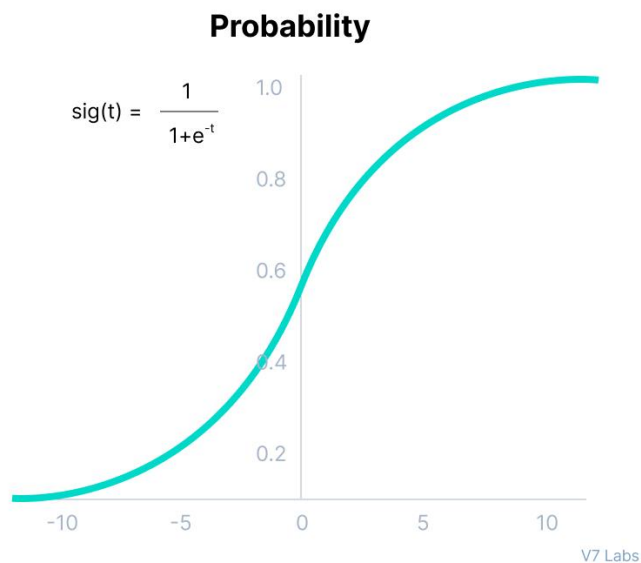


The limitations that this function faces include:

- The predictions may not be consistent for negative input values.
- The gradient for negative values is a small value that makes the learning of model parameters time-consuming.

Softmax Function

Before exploring the ins and outs of the Softmax activation function, we should focus on its building block—the sigmoid/logistic activation function that works on calculating probability values.



The output of the sigmoid function was in the range of 0 to 1, which can be thought of as probability.

But—

This function faces certain problems.

Let's suppose we have five output values of 0.8, 0.9, 0.7, 0.8, and 0.6, respectively. How can we move forward with it?

The answer is: We can't.

The above values don't make sense as the sum of all the classes/output probabilities should be equal to 1.

You see, the Softmax function is described as a combination of multiple sigmoids.

It calculates the relative probabilities. Similar to the sigmoid/logistic activation function, the SoftMax function returns the probability of each class.

It is most commonly used as an activation function for the last layer of the neural network in the case of multi-class classification.

Mathematically it can be represented as:

Softmax

$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$